

AWS Server and Database Migration

CLI-Based Approach

As-Built Document | Low Level Design (LLD)

Document Version: 1.0

Date: June 2026

Table of Contents

1. Overview and Prerequisites
2. AWS CLI Setup and Configuration
3. Server Migration using AWS DataSync
4. EC2 Instance Migration Strategy
5. Database Migration using DMS
6. Post-Migration Validation
7. Troubleshooting and Rollback

1. Overview and Prerequisites

1.1 Document Purpose

This document provides step-by-step instructions for migrating on-premises servers and databases to AWS using Command Line Interface (CLI) tools. It covers server migration using DataSync and database migration using AWS Database Migration Service (DMS).

1.2 Prerequisites

- AWS Account with appropriate IAM permissions
- AWS CLI v2 installed on source and management servers
- AWS DataSync agent deployed on-premises
- Source servers with SSH/RDP access for validation
- Adequate network bandwidth and VPN/Direct Connect setup
- S3 bucket for data transfer staging
- Target VPC and subnet configuration in AWS

1.3 Scope

- Physical or virtual servers (Windows/Linux)
- Relational databases (MySQL, PostgreSQL, SQL Server, Oracle)
- Application data and configuration files

2. AWS CLI Setup and Configuration

2.1 Install AWS CLI v2

For Linux/macOS:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip" unzip  
awscliv2.zip sudo ./aws/install
```

For Windows (PowerShell):

```
$ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -Uri  
https://awscli.amazonaws.com/AWSCLIV2.msi -OutFile AWSCLIV2.msi MSISetup.exe /I  
AWSCLIV2.msi /passive
```

2.2 Configure AWS Credentials

```
aws configure
```

When prompted, enter:

- AWS Access Key ID: [Your Access Key]
- AWS Secret Access Key: [Your Secret Key]
- Default region: us-east-1 (or your target region)
- Default output format: json

2.3 Verify Configuration

```
aws sts get-caller-identity
```

3. Server Migration using AWS DataSync

3.1 Deploy DataSync Agent On-Premises

8. Download DataSync agent from AWS Console
9. Deploy as VM on-premises (VMware, Hyper-V, or KVM)
10. Configure network connectivity (minimum 1Gbps recommended)
11. Note the agent IP address for CLI configuration

3.2 Create DataSync Location (On-Premises)

```
aws datasync create-location-nfs \ --subdirectory "/data" \ --server-hostname 10.0.1.50 \ --  
on-prem-config AgentArns=arn:aws:datasync:us-east-1:123456789012:agent/agent-12345678
```

Expected Output:

```
{  "LocationArn": "arn:aws:datasync:us-east-1:123456789012:location/nfs/12345678-1234-  
1234-1234-123456789012" }
```

3.3 Create S3 Location (Destination)

```
aws datasync create-location-s3 \ --s3-bucket-arn arn:aws:s3::migration-bucket \ --  
subdirectory /server-data \ --s3-config  
BucketAccessRoleArn=arn:aws:iam::123456789012:role/DataSyncS3Role
```

```
{  "LocationArn": "arn:aws:datasync:us-east-1:123456789012:location/s3/12345678-1234-  
1234-1234-123456789012" }
```

3.4 Create DataSync Task

```
aws datasync create-task \ --source-location-arn arn:aws:datasync:us-east-  
1:123456789012:location/nfs/source-location \ --destination-location-arn arn:aws:datasync:us-  
east-1:123456789012:location/s3/dest-location \ --name "Server-Migration-Task" \ --options  
TransferMode=CHANGED,Atime=BEST_EFFORT,Mtime=PRESERVE,VerifyMode=POINT_IN_  
TIME_CONSISTENT
```

```
{  "TaskArn": "arn:aws:datasync:us-east-1:123456789012:task/task-12345678" }
```

3.5 Start DataSync Task

```
aws datasync start-task-execution \ --task-arn arn:aws:datasync:us-east-  
1:123456789012:task/task-12345678
```

```
{  "TaskExecutionArn":  
"arn:aws:datasync:us-east-1:123456789012:task/task-12345678/execution/exec-12345678" }
```

3.6 Monitor Task Progress

```
aws datasync describe-task-execution \ --task-execution-arn arn:aws:datasync:us-east-  
1:123456789012:task/task-12345678/execution/exec-12345678
```

4. EC2 Instance Migration Strategy

4.1 Migrate Server Data to EBS Volume

Step 1: Create EBS Volume

```
aws ec2 create-volume \ --availability-zone us-east-1a \ --size 500 \ --volume-type gp3 \ --  
iops 3000 \ --throughput 125
```

```
{  "VolumeId": "vol-1234567890abcdef0",  "State": "creating" }
```

Step 2: Create EC2 Instance

```
aws ec2 run-instances \ --image-id ami-0c55b159cbfafa1f0 \ --instance-type m5.2xlarge \ --  
key-name migration-key \ --security-group-ids sg-12345678 \ --subnet-id subnet-12345678 \  
--block-device-mappings DeviceName=/dev/sda1,Ebs={VolumeSize=100,VolumeType=gp3}
```

```
{  "Instances": [    {      "InstanceId": "i-1234567890abcdef0",      "State": {"Name":  
"pending"}    }  ] }
```

Step 3: Attach EBS Volume to Instance

```
aws ec2 attach-volume \ --volume-id vol-1234567890abcdef0 \ --instance-id i-  
1234567890abcdef0 \ --device /dev/sdf
```

```
{  "State": "attaching",  "Device": "/dev/sdf" }
```

Step 4: Copy Data from S3 to EBS Volume

```
# SSH into EC2 instance ssh -i migration-key.pem ec2-user@ec2-instance-ip # Format and  
mount the volume sudo mkfs -t ext4 /dev/nvme1n1 sudo mkdir /mnt/data sudo mount  
/dev/nvme1n1 /mnt/data # Copy data from S3 aws s3 sync s3://migration-bucket/server-data  
/mnt/data --region us-east-1
```

4.7 Alternative: Manual VM Import/Export (Step-by-Step)

AWS DataSync (above) is the recommended approach for ongoing or large-scale server data migration. For a quick one-time migration of a whole VM — common for lab environments, small workloads, or a single legacy server — AWS also supports importing a VM directly as an EC2 instance from an exported OVA file. The walkthrough below follows the exact click-by-click and command-by-command steps.

Step 1 — Power off the source VM

In your hypervisor (VirtualBox, VMware, or Hyper-V), power off the VM completely — not pause, not save state.

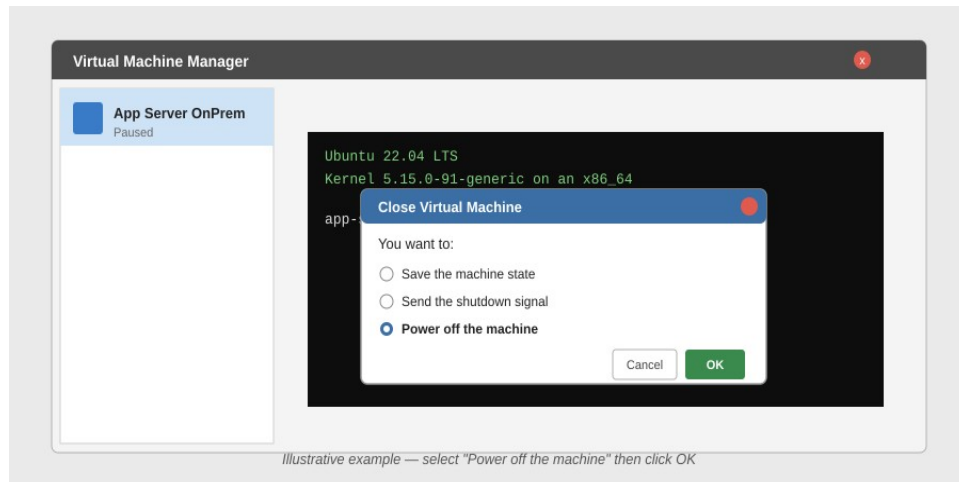


Figure 1 — Powering off the source VM

Step 2 — Export the VM to an OVA file

Right-click the powered-off VM and export it using the Open Virtualization Format (.ova). This bundles the disk and configuration into one file.

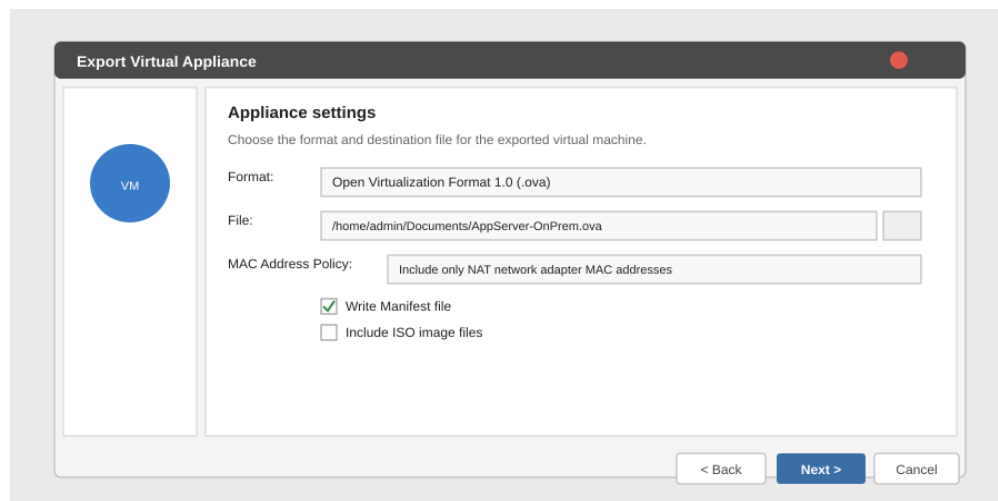


Figure 2 — Exporting the VM to .ova format

Step 3 — Create an S3 bucket and upload the OVA file

```
aws s3 mb s3://migration-bucket-2026 --region us-east-1
aws s3 cp AppServer-OnPrem.ova s3://migration-bucket-2026/
```

AWS Console

S3 > Buckets > migration-bucket-2026

migration-bucket-2026

Objects

Properties

Permissions

Upload

Name	Type	Size	Uploaded on
 AppServer-OnPrem.ova	ova file	8.2 GB	June 28, 2026
 trust-policy.json	json file	312 B	June 28, 2026



Upload succeeded

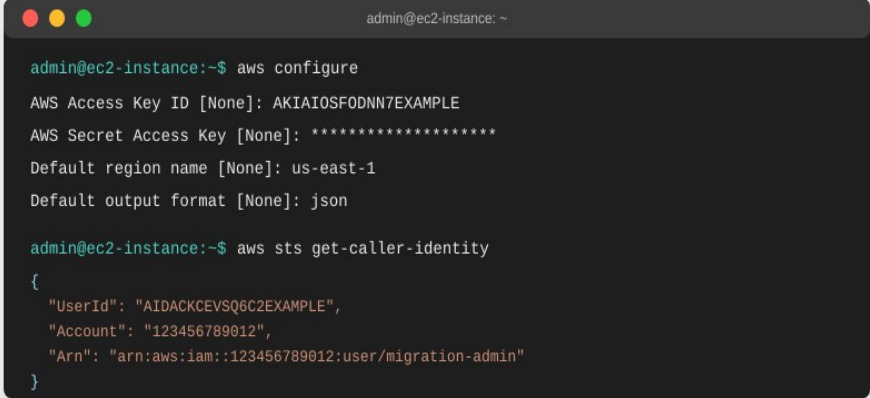
AppServer-OnPrem.ova was uploaded to migration-bucket-2026

Illustrative example — exported .ova file uploaded to an S3 bucket

Figure 3 — The .ova file after upload to S3

Step 4 — Configure the AWS CLI

Run `aws configure` with an IAM user that has EC2, S3, and IAM access, then confirm the identity.



```
admin@ec2-instance: ~  
  
admin@ec2-instance:~$ aws configure  
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE  
AWS Secret Access Key [None]: *****  
Default region name [None]: us-east-1  
Default output format [None]: json  
  
admin@ec2-instance:~$ aws sts get-caller-identity  
{  
  "UserId": "AIDACKCEVSQ6C2EXAMPLE",  
  "Account": "123456789012",  
  "Arn": "arn:aws:iam::123456789012:user/migration-admin"  
}
```

Illustrative example — values shown are sample placeholders, not real credentials

Figure 4 — Configuring the AWS CLI

Step 5 — Create the vmimport IAM role

`touch trust-policy.json`

```
{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Principal": { "Service":  
"vmie.amazonaws.com" }, "Action": "sts:AssumeRole", "Condition":  
{ "StringEquals": { "sts:Externalid": "vmimport" } } ] }
```

```
aws iam create-role --role-name vmimport \ --assume-role-policy-document "file://trust-  
policy.json" aws iam put-role-policy --role-name vmimport --policy-name vmimport \ --policy-  
document "file://role-policy.json"
```

Step 6 — Start the import

```
[ { "Description": "My Server OVA", "Format": "ova", "UserBucket": { "S3Bucket":  
"migration-bucket-2026", "S3Key": "AppServer-OnPrem.ova" } } ]
```

```
aws ec2 import-image --description "OnPrem App Server" \ --disk-containers "file://bucket.json"
```

Step 7 — Check import progress and launch the instance

```
aws ec2 describe-import-image-tasks --import-task-ids import-ami-052c4970ce867b739
```

```
admin@ec2-instance: ~  
  
admin@ec2-instance:~$ aws ec2 import-image \  
--description "OnPrem App Server" \  
--disk-containers "file://bucket.json"  
  
{  
  "Description": "OnPrem App Server",  
  "ImportTaskId": "import-ami-052c4970ce867b739",  
  "Progress": "1",  
  "Status": "active",  
  "StatusMessage": "pending"  
}  
  
admin@ec2-instance:~$ aws ec2 describe-import-image-tasks \  
--import-task-ids import-ami-052c4970ce867b739  
  
{  
  "Progress": "65",  
  "Status": "active",  
  "StatusMessage": "converting"  
}
```

Illustrative example — task IDs and progress values are samples

Figure 5 — Tracking the import task until it completes

Once the StatusMessage reads completed, a new AMI appears in EC2 > Images > AMIs. Launch an instance from it the same way as any other AMI.

AWS Console EC2 > Instances

Instances (1) Connect

Name	Instance ID	Instance state	Instance type	Status check
☐ migrated-app-server	i-0a1b2c3d4e5f6g7h8	● Running	m5.2xlarge	● 2/2 checks passed

Instance summary

Public IPv4 address 54.210.18.92	Private IP address 10.0.1.45	AMI ID ami-0c9d8e7f6a5b4c3d2	Launch time June 28, 2026
---	---------------------------------	---	------------------------------

Imported successfully from on-premises OVA — migration complete

Illustrative example — the imported VM now runs as a native EC2 instance

Figure 6 — The migrated server running as an EC2 instance

Try it yourself

12. Create a VM in VirtualBox, VMware, or Hyper-V running Ubuntu
13. Deploy a simple web application and a MySQL database on it
14. Follow Steps 1-7 above to migrate that VM into AWS

5. Database Migration using AWS DMS

5.1 Create DMS Replication Instance

```
aws dms create-replication-instance \ --replication-instance-identifier db-migration-instance \
--replication-instance-class dms.c5.2xlarge \ --allocated-storage 500 \ --engine-version "3.4.7"
\ --multi-az \ --publicly-accessible false \ --vpc-security-group-ids sg-12345678 \ --db-
subnet-group-name dms-subnet-group \ --availability-zone us-east-1a
```

```
{  "ReplicationInstance": {      "ReplicationInstanceIdentifier": "db-migration-instance",
"ReplicationInstanceArn": "arn:aws:dms:us-east-1:123456789012:rep:12345678",
"ReplicationInstanceStatus": "creating"  } }
```

5.2 Create Source Database Endpoint

For MySQL/PostgreSQL:

```
aws dms create-endpoint \ --endpoint-identifier source-mysql-db \ --endpoint-type source \ --
engine-name mysql \ --server-name 10.0.1.100 \ --port 3306 \ --database-name
production_db \ --username admin \ --password 'YourPassword123!' \ --tags
Key=Environment,Value=Migration Key=Type,Value=Source
```

```
{  "Endpoint": {      "EndpointIdentifier": "source-mysql-db",      "EndpointArn":
"arn:aws:dms:us-east-1:123456789012:ep:12345678",      "Status": "creating"  } }
```

5.3 Create Target Database Endpoint (RDS)

```
aws dms create-endpoint \ --endpoint-identifier target-rds-mysql \ --endpoint-type target \ --
engine-name mysql \ --server-name myrds-instance.123456789012.us-east-
1.rds.amazonaws.com \ --port 3306 \ --database-name production_db \ --username admin \
--password 'YourPassword123!' \ --tags Key=Environment,Value=Migration
Key=Type,Value=Target
```

5.4 Test Endpoint Connection

```
aws dms test-connection \ --replication-instance-arn arn:aws:dms:us-east-
1:123456789012:rep:12345678 \ --endpoint-arn arn:aws:dms:us-east-
1:123456789012:ep:source-mysql-db
```

5.5 Create Migration Task

```
aws dms create-replication-task \ --replication-task-identifier mysql-to-rds-migration \ --
source-endpoint-arn arn:aws:dms:us-east-1:123456789012:ep:source-mysql-db \ --target-
endpoint-arn arn:aws:dms:us-east-1:123456789012:ep:target-rds-mysql \ --replication-
instance-arn arn:aws:dms:us-east-1:123456789012:rep:12345678 \ --migration-type cdc \ --
table-mappings file://table-mappings.json \ --replication-task-settings file://task-settings.json
```

Sample table-mappings.json:

```
{  "rules": [      {      "rule-type": "selection",      "rule-id": "1",      "rule-name": "include-all-tables",
"object-locator": {          "schema-name": "%",          "table-name": "%"      },      "rule-action":
"include"  }  ] }
```

5.6 Start Migration Task

```
aws dms start-replication-task \ --replication-task-arn arn:aws:dms:us-east-1:123456789012:task:12345678 \ --start-replication-task-type start-replication
```

Options for start-replication-task-type:

- start-replication: Full load + CDC (recommended)
- resume-processing: Resume interrupted migration
- reload-target: Reload target with fresh full load

5.7 Monitor Migration Progress

```
aws dms describe-replication-tasks \ --filters Name=replication-task-arn,Values=arn:aws:dms:us-east-1:123456789012:task:12345678
```

Check specific table statistics:

```
aws dms describe-table-statistics \ --replication-task-arn arn:aws:dms:us-east-1:123456789012:task:12345678
```

6. Post-Migration Validation

6.1 Validate EC2 Instance

```
# Check instance status aws ec2 describe-instance-status \ --instance-ids i-1234567890abcdef0 \ --query 'InstanceStatuses[0].{InstanceState:InstanceState.Name,SystemStatus:SystemStatus.Status}'
# Verify data integrity aws s3api head-bucket --bucket migration-bucket aws s3 ls s3://migration-bucket/server-data --recursive --summarize
```

6.2 Validate Database Migration

```
# Get RDS instance endpoint aws rds describe-db-instances \ --db-instance-identifier mydb-instance \ --query 'DBInstances[0].Endpoint.Address'
# Connect to target RDS and verify data mysql -h mydb-instance.123456789012.us-east-1.rds.amazonaws.com -u admin -p MySQL> SELECT COUNT(*) FROM production_db.users;
MySQL> SELECT COUNT(*) FROM production_db.orders;
```

6.3 Row Count Validation Script

```
#!/bin/bash # Compare row counts between source and target SOURCE_DB="10.0.1.100"
TARGET_DB="mydb-instance.123456789012.us-east-1.rds.amazonaws.com" echo "Validating table row counts..." TABLES=(users orders products transactions) for table in "${TABLES[@]"; do SOURCE_ROWS=$(mysql -h $SOURCE_DB -u admin -p$SOURCE_PWD production_db -se "SELECT COUNT(*) FROM $table") TARGET_ROWS=$(mysql -h $TARGET_DB -u admin -p$TARGET_PWD production_db -se "SELECT COUNT(*) FROM $table") if [ "$SOURCE_ROWS" == "$TARGET_ROWS" ]; then echo "✓ $table: $SOURCE_ROWS rows (MATCH)" else echo "✗ $table: Source=$SOURCE_ROWS, Target=$TARGET_ROWS (MISMATCH)" fi done
```

6.4 Performance Testing

```
# Test RDS performance aws cloudwatch get-metric-statistics \ --namespace AWS/RDS \ --metric-name DatabaseConnections \ --dimensions Name=DBInstanceIdentifier,Value=mydb-instance \ --start-time 2024-01-01T00:00:00Z \ --end-time 2024-01-01T01:00:00Z \ --period 300 \ --statistics Average,Maximum
```

7. Troubleshooting and Rollback

7.1 Common Issues and Solutions

Issue	Cause	Solution
Network timeout	Source/Target unreachable	Check security groups, VPN, Direct Connect connectivity
Authentication error	Invalid credentials	Verify database user/password, IAM permissions
Disk full	Replication instance out of space	Increase replication instance storage or reduce batch size
Data mismatch	Incomplete replication/CDC lag	Wait for CDC to catch up, verify binlog settings

7.2 View DMS Task Errors

```
aws dms describe-replication-tasks \ --filters Name=replication-task-arn,Values=arn:aws:dms:us-east-1:123456789012:task:12345678 \ --query 'ReplicationTasks[0].{Status:Status,LastFailureMessage:LastFailureMessage}'
```

7.3 Rollback Procedure

15. Stop DMS Task:

```
aws dms stop-replication-task \ --replication-task-arn arn:aws:dms:us-east-1:123456789012:task:12345678
```

16. Keep source database running as primary

17. Delete failed RDS instances:

```
aws rds delete-db-instance \ --db-instance-identifier mydb-instance \ --skip-final-snapshot
```

18. Terminate EC2 instances:

```
aws ec2 terminate-instances --instance-ids i-1234567890abcdef0
```

19. Redirect application traffic back to source